

口罩佩戴检测-实训报告

2312966 林晖鹏

一、问题重述

题目背景：

今年一场席卷全球的新型冠状病毒给人们带来了沉重的生命财产的损失。有效防御这种传染病病毒的方法就是积极佩戴口罩。我国对此也采取了严肃的措施，在公共场合要求人们必须佩戴口罩。

在本次实验中，我们要建立一个目标检测的模型，可以识别图中的人是否佩戴了口罩。

作业要求：

- 建立深度学习模型，检测出图中的人是否佩戴了口罩，并将其尽可能调整到最佳状态。
- 学习经典的模型 MTCNN 和 MobileNet 的结构。
- 学习训练时的方法。

二、设计思想及主体代码内容

本次实验主要是学会搭建并训练MobileNet模型，从而实现口罩佩戴检测的任务。训练的流程如下图。



本实验流程图

实验中的代码框架都提供了，这里主要是调整部分参数。调整的参数主要有三个：**epochs**、**lr**、**factor**和**patiences**。

- **epochs**: 训练轮次。
- **lr**: 初始学习率，如果一开始模型损失下降较慢，可以考虑增大初始学习率。
- **factor**: 当学习率需要衰减的时候， $lr = lr * factor$ 。
- **patiences**: 容忍度。当patiences个批次训练之后模型损失没有下降，就衰减学习率。

下面是我最终调参的结果参数。

代码块

```
1  epochs = 40
2  model = MobileNetV1(classes=2).to(device)
3  optimizer = optim.Adam(model.parameters(), lr=1e-3) # 优化器
4  # optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
5  # 学习率下降的方式，acc三次不下降就下降学习率继续训练，衰减学习率
6  scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer,
7                                                    'max',
8                                                    factor=0.3,
9                                                    patience=12)
10 # 损失函数
11 criterion = nn.CrossEntropyLoss()
```

调参过程

1. epochs

一开始尝试轮次为100，发现训练集的损失函数能下降到0.005，但是明显过拟合了，测试得分只有50分。所以我们将训练轮次调整下降到30-50尝试。

2. factor

当factor为0.5的时候，多次训练查看训练日志都发现，模型的损失率无法进一步下降，可能是在后期学习率的步长太大了，这里学习率衰减太慢了。所以我们考虑把衰减因子调整到0.2-0.3。

3. patience

容忍度太大会导致学习率难以衰减，容忍度太小又会导致学习率过早变小导致损失函数下降太慢。所以我们要合理设置容忍度。这里考虑我们的批次为32批次，训练轮次为40左右，我们设置容忍度为10-13。

4. 优化器

除了原有的优化器，我还尝试了AdamW这个优化器，但是效果还不如原有的，额外查阅了其他优化器，感觉都不如原有的Adam适用于这个题目。

5. 观察loss函数变化

在调参过程中，很实用的一个方法是观察损失函数的变化。

如果损失函数跳动比较大，可能是学习率的步长太大了，此时需要减小容忍度和增大学习率衰减因子。

如果损失函数下降太慢了，到训练结束都是下降的趋势，说明学习率太小了。可能是初始学习率太小，也可能是容忍度太小和衰减因子太大导致学习率过早下降。

通过这些技巧我们来调整上面的几个参数。

6. 随机性

这里只固定了测试集训练集划分的随机种子，但是对其他随机数是没有固定随机种子的，所以这里的训练过程不可复现，导致后来我本来训练到一个满分的模型之后，重新训练一次之后又不能满分了。但同时说明我这里的参数调的还不够好，有待优化。

三、实验结果

最终代码

代码块

```
1  # 1.加载数据并进行数据处理
2
3  data_path = "./datasets/5f680a696ec9b83bb0037081-momodel/data/"
4  # 加载 MobileNet 的预训练模型权
5  device = torch.device("cuda:0") if torch.cuda.is_available() else
    torch.device("cpu")
6  train_data_loader, valid_data_loader = processing_data(data_path=data_path,
    height=160, width=160, batch_size=32)
7  modify_x, modify_y = torch.ones((32, 3, 160, 160)), torch.ones((32))
8
9
10 # 2.如果有预训练模型，则加载预训练模型；如果没有则不需要加载
11
12 # 3.创建模型和训练模型，训练模型时尽量将模型保存在 results 文件夹
13 epochs = 40
14 model = MobileNetV1(classes=2).to(device)
15
16 # optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
17 optimizer = optim.Adam(model.parameters(), lr=1e-3) # 优化器
18
19 # 学习率下降的方式，acc三次不下降就下降学习率继续训练，衰减学习率
20 scheduler =
    optim.lr_scheduler.ReduceLROnPlateau(optimizer, 'max', factor=0.3, patience=12)
```

```

21
22 # 损失函数
23 criterion = nn.CrossEntropyLoss()
24
25 best_loss = 1e9
26 best_model_weights = copy.deepcopy(model.state_dict())
27 loss_list = [] # 存储损失函数值
28 for epoch in range(epochs):
29     model.train()
30
31     for batch_idx, (x, y) in tqdm(enumerate(train_data_loader, 1)):
32         x = x.to(device)
33         y = y.to(device)
34         pred_y = model(x)
35
36         # print(pred_y.shape)
37         # print(y.shape)
38
39         loss = criterion(pred_y, y)
40         optimizer.zero_grad()
41         loss.backward()
42         optimizer.step()
43
44         if loss < best_loss:
45             best_model_weights = copy.deepcopy(model.state_dict())
46             best_loss = loss
47
48         loss_list.append(loss)
49
50     print('step:' + str(epoch + 1) + '/' + str(epochs) + ' || Total Loss:
%.4f' % (loss))
51     torch.save(model.state_dict(), './results/new1.pth')
52
53 # 4.评估模型，将自己认为最佳模型保存在 result 文件夹，其余模型备份在项目中其它文件夹，方
便您加快测试通过。
54
55 # model.eval()
56 # correct = 0
57 # total = 0
58 # with torch.no_grad():
59 #     for val_x, val_y in valid_data_loader:
60 #         val_x, val_y = val_x.to(device), val_y.to(device)
61 #         outputs = model(val_x)
62 #         _, predicted = torch.max(outputs.data, 1)
63 #         total += val_y.size(0)
64 #         correct += (predicted == val_y).sum().item()
65 # val_acc = correct / total

```

```
66 # print(f"验证准确率: {val_acc:.4f}")
67 # scheduler.step(val_acc)
68
69 plt.plot(loss_list, label = "loss")
70 plt.legend()
71 plt.savefig("loss_plot.png") # 保存为PNG文件
72 plt.show()
73
```

最终我们得到了满昏!!!

测试详情

测试点	状态	时长	结果
在 5 张图片上 测试模型	✓	4s	得分:100.0

确定

四、总结

- 本次实验我们主要是了解MobileNet的训练流程，并且尝试调整参数来训练较好的模型。
- 主要的任务是调参，在实验过程中，对调参的方法有了进一步的理解。什么时候需要调参？什么情况下该调哪些参数？往大往小调？
- 该模型还有v2v3版本，我们还可以去尝试更好的模型，以达到更好的效果。
- 同时，进一步熟悉了模型训练的代码框架。